

* Always Remember: → Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. — vHixen

* Problem: —

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position. Return the minimum number of jumps required to reach the last index ($n - 1$).

Input: `nums = [2, 3, 1, 1, 4]`

Output: 2

↪ Top

→ If array size is n then top is $n - 1$.

* Prerequisites (Very Important):-

- This question involves to explore multiple options / possibilities, means number of options are not fixed.
- So Loops are used in such questions, To identify how to use Loops in such questions, Watch Lecture 10 - 1D DP and then comeback here.
- In that, I have provided deep view on how to identify loops.

* How Question provides hints to think Recursive?

- (i) Tries to give you options to explore
- (ii) Tells you can't go greedy, must explore all possibilities/options to build result

* Recursive code:- (LL - Jump Game)

bool Recursion(i, nums) {

if (i == n-1) return true

if (i >= n) return false

int start = i+1

int end = i + nums[i]

for (int j = start; j <= end; j++) {

// Jump from i to jth index

// we want to find can reach index n-1

// so call Recursion(j), do the same

bool canReachEndFrom_j = Recursion(j, nums)

if (canReachEndFrom_j) return true;

}

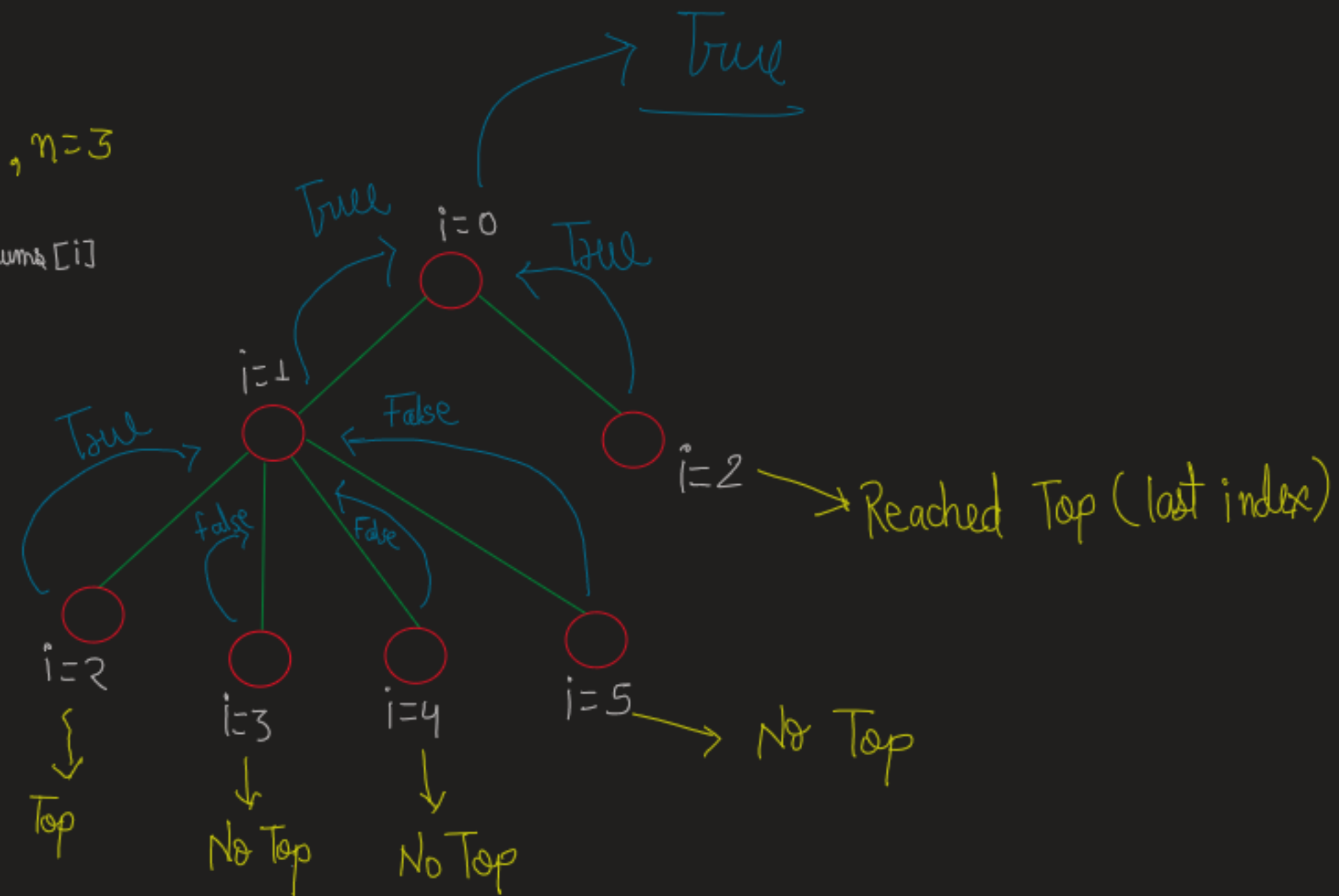
{ return false

nums =

2	4	1
---	---	---

, n=3

start = i+1
end = i + nums[i]

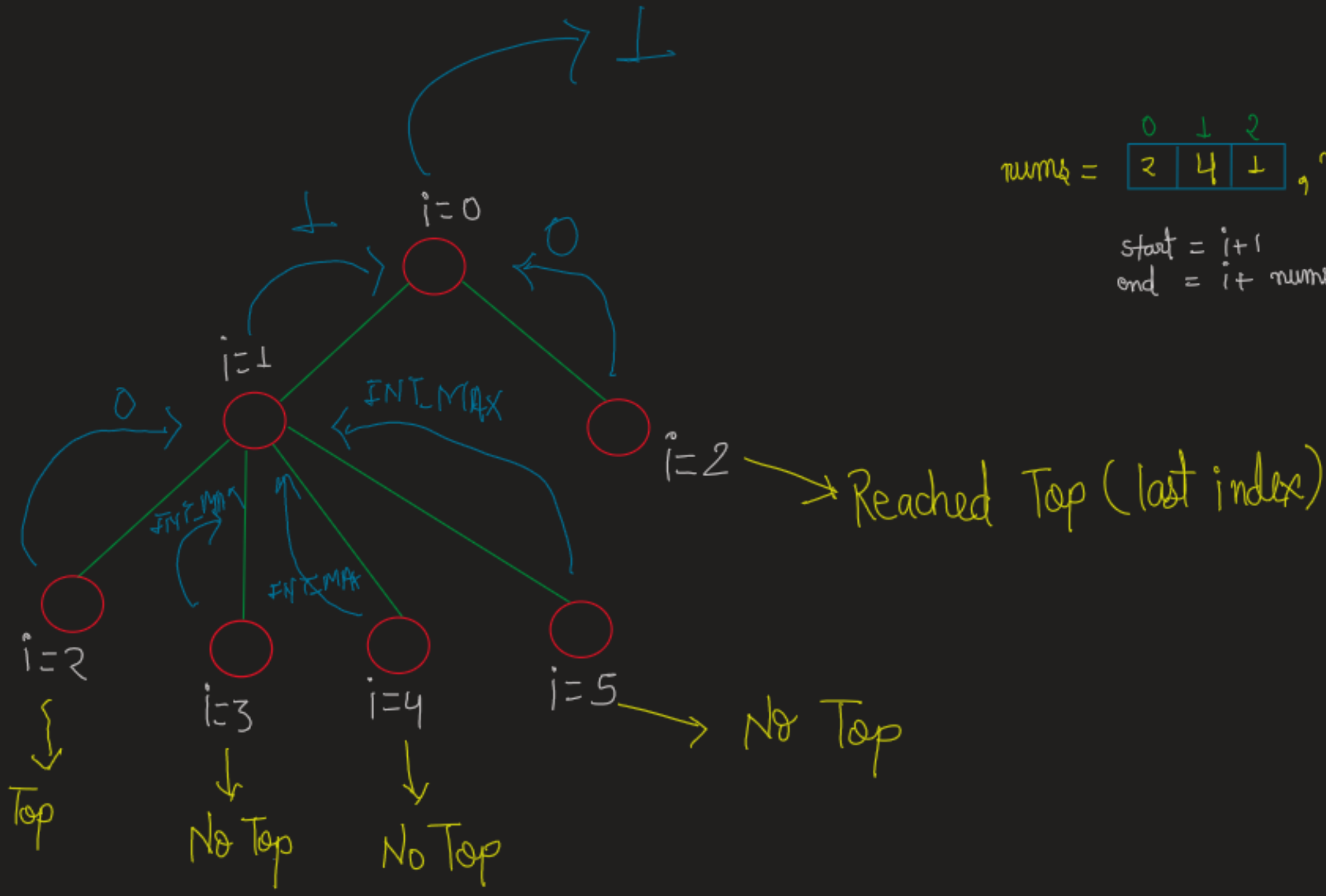


nums =

2	4	1
---	---	---

, n = 3

start = i + 1
end = i + nums[i]



→ If reached top ($i == n - 1$) return 0 as Indication.

→ If ($i > n$) return INT_MAX as Indication that we can't reach Top / Last index.

• Base cases are clear now.

* Recursive code:-

```
int Recursion(i, nums) {
```

```
    if (i == n-1) return 0
```

```
    if (i == n) return INT_MAX
```

```
    int minJumps = INT_MAX
```

```
    int start = i+1
```

```
    int end = i + nums[i]
```

```
    for (int j = start; j <= end; j++) {
```

```
        int JumpsTakenFrom-j = Recursion(j, nums);
```

```
        if (JumpsTakenFrom-j != INT_MAX) { // means we can reach n-1 from j
```

```
            int curPathJumps = JumpsTakenFrom-j + 1;
```

```
            minJumps = min(minJumps, curPathJumps);
```

```
        }
```

```
    }
```

```
    return minJumps;
```

```
}
```

nums =

0	1	2
2	4	1

, n=3

start = i+1
end = i + nums[i]

